

Simon Says FPGA Game Final Report

EE354 Final Project

Nexys A7 FPGA Board

Spring 2026

Team Members: Steven Zhou and Isais

Written Report

Table of Contents

- 1. Introduction
- 2. Background
- 3. Design
 - 3.1 Overall Architecture
 - 3.2 State Machine / Architecture
- 4. Implementation Details
- 5. Testing and Verification
- 6. User Interface
- 7. Challenges
- 8. Future Work
- 9. Conclusion
- 10. References
- Appendix A. Main Source Files

1. Introduction

For our EE354 final project, we built a digital version of the classic Simon Says memory game on the Nexys A7 FPGA board. The main idea is straightforward: the board shows the player a sequence of colors, and the player repeats that same sequence using the board buttons. Each time the player gets the sequence right, the game adds one more color to the pattern, so the rounds become harder as the game continues.

We chose this project because it uses a lot of the material from the class in one complete system. The game needed a finite state machine, button input, timing counters, sequence storage, LED output, VGA graphics, and seven-segment display control. It also gave us a chance to test something interactive on the actual FPGA board instead of only checking waveforms in simulation.

The final design uses the Nexys A7 push buttons for player input, the LEDs and VGA monitor for visual feedback, SW0 as a difficulty switch, and the seven-segment display to show the current round and high score. The player wins by completing 10 rounds and loses immediately after entering a wrong color. This follows the original project plan, but the final version also includes some extra details, such as RGB LED status feedback and a cleaner two-state timing approach for showing each color.

2. Background

The most important concept behind the project is the finite state machine. Since the game moves through different stages, an FSM was the cleanest way to organize the design. The circuit needs one behavior when it is idle, another behavior when it is showing the sequence, another behavior when it is waiting for player input, and separate behaviors when the player wins or loses.

The design is also based on synchronous sequential logic. The main registers update on the rising edge of the clock. This includes the current state, current round, player input index, sequence display index, timer value, stored sequence, and high score. Keeping these values clocked made the game easier to control and easier to debug.

Button input was another major part of the project. Mechanical buttons do not switch cleanly every time they are pressed. They can bounce for a short time, which means one physical press can look like several fast transitions. If the game counted those transitions directly, one button press could accidentally become multiple inputs. To prevent this, every button is passed through debounce and one-pulse logic before the game state machine uses it.

For the color pattern, the design uses an LFSR, or linear feedback shift register. The LFSR generates pseudo-random values, and the lower two bits are used to select one of the four game colors. This was a simple hardware-friendly way to create a changing Simon Says sequence.

The project also uses VGA output so the game is easier to see and play. The VGA screen is divided into four large colored quadrants: red, blue, green, and yellow. During the show sequence, the active color becomes brighter, while the other colors stay dim. This gives the player a clear visual cue that matches the LEDs on the board.

3. Design

3.1 Overall Architecture

The design is split into several Verilog modules instead of putting all of the logic into one large file. This made the project easier to build, test, and fix. The top-level module connects the button input block, game state machine, VGA timing block, VGA renderer, seven-segment display driver, LED outputs, and RGB LED indicators.

Module	Purpose
simon_says_top.v	Top-level file that connects the full design together.
simon_game_sm.v	Main game state machine, sequence memory, LFSR, round logic, and LED drive.
button_input.v	Converts raw board buttons into clean start and player input signals.
debounce_onepulse.v	Synchronizes and debounces a button, then creates a one-clock pulse.
vga_timing_640x480.v	Generates VGA timing, pixel coordinates, active-video, and sync signals.
simon_vga_renderer.v	Draws the four colored quadrants, status banner, and score digits.
seven_seg_digit_pixel.v	Draws seven-segment-style digits in the VGA renderer.
simon_ssd_display.v	Builds the physical SSD digit values for round and high score.
ssd_mux.v	Multiplexes the eight-digit seven-segment display.

The board controls are mapped so that the buttons match the position of the colored regions on the VGA screen. This made the game easier to understand during the demo because each input button lines up with its on-screen quadrant.

Button	Game Color	Screen Location
BTNU	Red	Top-left
BTNR	Blue	Top-right
BTNL	Green	Bottom-left
BTND	Yellow	Bottom-right
BTNC	Start / Restart	Center button

3.2 State Machine / Architecture

The main game control happens inside `simon_game_sm.v`. In the proposal, the game was described using the main stages IDLE, SHOW SEQUENCE, WAIT FOR INPUT, CHECK INPUT, and GAME OVER / YOU WIN. In the final code, the same general flow is still used, but the show sequence was split into two states: one state where the color is on and one state where the color is off. That small change made the timing cleaner and easier to test.

State	Description
ST_IDLE	Waits for the player to press the start button.
ST_SHOW_ON	Turns on the current color in the stored sequence.

State	Description
ST_SHOW_OFF	Turns the color off between sequence flashes.
ST_WAIT	Waits for the player to enter the sequence with the color buttons.
ST_LOSE	Shows the losing condition after a wrong input.
ST_WIN	Shows the winning condition after all rounds are completed.

The game starts in ST_IDLE. When the player presses BTNC, the game initializes round 1, generates the first sequence value, and moves into the show sequence. The FSM alternates between ST_SHOW_ON and ST_SHOW_OFF until the whole sequence for that round has been displayed. After that, the game enters ST_WAIT and accepts player input.

While the game is in ST_WAIT, each valid button press is compared against the stored sequence at the current input index. If the input is correct and the player still has more colors to enter, the input index advances. If the player finishes the whole round correctly, the round increases and a new color is added to the sequence. If the player enters the wrong color, the FSM moves to ST_LOSE. If the player completes the final round, the FSM moves to ST_WIN.

Transition	Condition
ST_IDLE to ST_SHOW_ON	Start pulse received.
ST_SHOW_ON to ST_SHOW_OFF	Flash-on timer expires.
ST_SHOW_OFF to ST_SHOW_ON	More colors still need to be shown.
ST_SHOW_OFF to ST_WAIT	The full current-round sequence has been shown.
ST_WAIT to ST_SHOW_ON	Player completes the round correctly.
ST_WAIT to ST_LOSE	Player presses the wrong color.
ST_WAIT to ST_WIN	Player completes the maximum round.
ST_LOSE or ST_WIN to ST_SHOW_ON	Start button is pressed again to restart.

4. Implementation Details

4.1 Button Input and Debouncing

The button_input module receives the raw button signals from the board. The start button and each color button are passed through an instance of debounce_onepulse. The debounce module first synchronizes the button to the clock, waits for the signal to remain stable, and then produces a clean button level. It also generates a one-clock-cycle pulse on the rising edge of a stable press.

This one-pulse output is important because the game should count one button press only once. If the player holds a button down, the FSM should not keep receiving new inputs every clock cycle. The color pulses are then encoded into a two-bit player_value signal, and player_valid tells the game state machine that the value should be checked.

Encoded Value	Color
2'd0	Red
2'd1	Blue
2'd2	Green
2'd3	Yellow

4.2 Sequence Generation

The color sequence is stored in an internal array inside the game state machine. Each entry is a two-bit value that represents one of the four colors. To generate new colors, the design uses an 8-bit LFSR with a seed value of 8'hA5. The feedback is made by XORing selected bits, then shifting the register. The lower two bits of the next LFSR value are used as the next color symbol.

This is not truly random, but it works well for a small FPGA game. It gives the player a changing pattern while staying simple enough to implement with a few registers and XOR logic.

4.3 Round, High Score, and Difficulty

The round register starts at 1 after the player presses start. Every time the player completes the current sequence correctly, the game advances to the next round and appends one new color. If the player reaches the maximum round and completes it, the game enters the win state.

The high score register stores the highest round reached during the current session. The high score is updated when the player completes a round that is higher than the previous saved value. If the player wins, the high score is set to the maximum round value.

SW0 is used as the difficulty switch. When SW0 is low, the design uses the slower show timing values. When SW0 is high, it uses faster timing values. This made it easy to add a difficulty option without changing the rest of the FSM.

4.4 LED and RGB LED Output

The four basic LEDs mirror the active Simon Says color. During the show sequence, only the LED for the active color turns on. During the input state, the LEDs also provide short feedback after the player presses a button. In the lose state, LED0 turns on. In the win state, all four LEDs turn on.

The top-level module also drives two RGB LEDs. LD16 shows the active color whenever a pad is lit, so the active game color is visible even without a monitor. LD17 acts as a state indicator and mirrors the VGA banner color: idle, show, input, lose, or win.

4.5 VGA Output

The VGA display uses a 640 by 480 layout. The `vga_timing_640x480` module generates the horizontal and vertical counters, `hsync`, `vsync`, `active_video`, `pixel_tick`, and `frame_tick` signals. The screen is divided into four 320 by 240 quadrants. Each quadrant corresponds to one game color.

Quadrant	Color
Top-left	Red
Top-right	Blue
Bottom-left	Green
Bottom-right	Yellow

The `simon_vga_renderer` module decides the RGB output based on the current pixel location and game mode. Inactive quadrants are drawn dim, while the active quadrant is drawn brighter when `light_on` is asserted. The renderer also draws a banner at the top of the screen. The banner changes color based on whether the game is idle, showing the sequence, waiting for input, in the lose state, or in the win state.

The VGA screen also includes seven-segment-style score digits for the current round and high score. This made the VGA output more useful because the player can see the game board and the score information in the same place.

4.6 Seven-Segment Display

The physical seven-segment display shows the current round and high score. The `simon_ssd_display` module converts the round and high-score values into digit values. The `ssd_mux` module then cycles through the eight digits quickly so that the display appears steady. Blank values are used for unused digits, so the display is easier to read.

5. Testing and Verification

We tested the project in simulation and on the Nexys A7 board. The main simulation files were `simon_game_sm_tb.v` and `simon_says_top_tb.v`. The state-machine testbench focused on the game logic by itself, while the top-level testbench checked the design after the major modules were connected together.

For simulation, the timing parameters were reduced so that the tests would run quickly. The actual hardware timing values are much larger because the player needs enough time to see each flash. Using smaller values in simulation let us test the same behavior without waiting through long real-time delays.

Test Area	What Was Checked
Reset behavior	The game returns to idle and clears the round after reset.
Start button	Pressing BTNC starts or restarts the game.
Debouncing	Button bounce does not create extra valid inputs.
One-pulse logic	One stable press creates one pulse.
Sequence display	The active LED and VGA quadrant match the stored sequence.
Correct input	Correct player input advances the index or next round.
Wrong input	Wrong player input moves the FSM to the lose state.
Win condition	Completing the maximum round moves the FSM to the win state.

Test Area	What Was Checked
Difficulty switch	SW0 changes the flash timing.
SSD output	The physical display shows round and high score values.
VGA output	Quadrants, brightness, banner colors, and score digits display correctly.

Hardware testing was also necessary because some issues are hard to see in simulation. Button bounce, VGA readability, LED brightness, and overall gameplay feel are better tested on the actual board. After the design was loaded onto the Nexys A7, we checked that the buttons matched the correct colors, the LEDs matched the screen, the SSD updated properly, and the game could be restarted after win or loss.

6. User Interface

The user interface is meant to be simple. The player presses BTNC to start the game. The board then flashes the sequence using both LEDs and VGA. After the sequence is shown, the player repeats the sequence with the four directional buttons. If the input is correct, the game moves to the next round. If the input is wrong, the game immediately goes to the game over state. If the player completes all 10 rounds, the game goes to the win state.

The VGA screen makes the game easier to follow because each color has a large region on the screen. The LEDs provide extra feedback on the board, and the seven-segment display shows the current round and high score. SW0 lets the player switch between slow and fast mode. Overall, the interface is simple enough that someone can understand the game without needing to know how the Verilog works.

7. Challenges

One of the biggest challenges was getting the timing of the show sequence to feel right. At first, it seemed like one show state would be enough, but it became easier to control the sequence after splitting it into ST_SHOW_ON and ST_SHOW_OFF. That made each flash and pause easier to manage.

Button input was another challenge. Without debouncing, a normal button press could accidentally be counted more than once. This could make the player lose even when they pressed the correct button. Using a reusable debounce and one-pulse module fixed that problem.

The VGA display was also challenging because the renderer had to combine several things at once: quadrant color, active brightness, the top banner, score digits, and blanking outside the visible region. We kept the graphics simple so the design would stay reliable and readable.

Another challenge was testing. Simulation needed short timing values, but hardware needed long timing values. We handled this by parameterizing the timing values and using smaller numbers in the testbenches. This let us verify the game logic quickly before testing the final version on the board.

The last major challenge was integration. Each module could be tested on its own, but the complete project only worked if all signal mappings matched. The button encoding had to match the FSM, the FSM display modes had to match the VGA renderer, and the score values had to match both the VGA digits and physical SSD.

8. Future Work

One feature we would add in the future is sound. The original Simon Says game has a different tone for each color, so adding a buzzer or speaker output would make the FPGA version feel more complete.

Another improvement would be more detailed VGA text. The current design uses colors, banners, and score digits. A future version could include larger text such as START, INPUT, GAME OVER, and YOU WIN using a character ROM.

The difficulty system could also be expanded. Instead of only slow and fast mode, the switches could select multiple difficulty levels. These modes could change the flash speed, the pause length, or the number of rounds needed to win.

A better random seed would also improve the game. Right now, the LFSR starts from a fixed seed. In a future version, the design could use the amount of time before the player presses start to create a different seed each time.

The high score could also be improved. The current high score is stored during the active session. A future design could save the high score in a way that survives reset or power cycling.

9. Conclusion

For this project, we built a working Simon Says game on the Nexys A7 FPGA board. The final design includes a game state machine, LFSR-based sequence generation, debounced button input, LED feedback, RGB LED feedback, VGA graphics, seven-segment score display, and a difficulty switch.

The project helped connect several EE354 topics into one complete system. Instead of only writing small independent modules, we had to make the full design work together. The FSM controlled the game flow, the input modules made the buttons reliable, the VGA modules handled the display, and the SSD modules showed the score.

Overall, the project met our original goals. The game can start, show a sequence, accept player input, check the input, advance rounds, track a high score, and display win or loss states. It was a useful example of how digital logic and Verilog can be used to build an interactive hardware project.

10. References

Simon Says FPGA Project Proposal, EE354 Final Project, Spring 2026.

Simon Says EE354 Final Project presentation slides, Spring 2026.

Nexys A7 board constraint files and Verilog source files used for final integration and testing.

Appendix A. Main Source Files

The final project used the following main source files: `simon_says_top.v`, `simon_game_sm.v`, `button_input.v`, `debounce_onepulse.v`, `vga_timing_640x480.v`, `simon_vga_renderer.v`, `seven_seg_digit_pixel.v`, `simon_ssd_display.v`, `ssd_mux.v`, `simon_game_sm_tb.v`, `simon_says_top_tb.v`, and `nexys_a7_simon_says.xdc`.